

Information Retrieval: Das Vektorraummodell



Ingo Schmitt

Brandenburgische Universität Cottbus

Institut für Informatik

Lehrstuhl für Datenbank- und Informationssysteme



Überblick

- am häufigsten eingesetztes Modell
- Dokumente und Anfrage repräsentiert durch t -dimensionale Vektoren
- t ist Termanzahl des Indexierungsvokabulars
- i -te Vektorkomponente ist Termgewicht (reelle Zahl)
- Hohes Gewicht $\Rightarrow$ entsprechender Term repräsentiert Dokument/Anfrage gut



5.1 Grundidee

- N sei Anzahl Dokumente
- n_k sei Anzahl Dokumente, die Term k enthalten
- tf_{dk} sei Vorkommenshäufigkeit von Term k in Dokument D
- Dokument D wird durch Vektor $V_D = (w_{d1}, \dots, w_{dt})$ repräsentiert
- Analog Anfrage Q durch $V_Q = (w_{q1}, \dots, w_{qt})$
- w_{qk} bzw. w_{dk} sind Relevanz von Dokument D bzw. Anfrage Q für Term k
- Ähnlichkeitsberechnung mittels Skalarprodukt: $\langle V_Q, V_D \rangle$

$$\text{similarity}(V_Q, V_D) = \langle V_Q, V_D \rangle = \sum_{k=1}^t w_{qk} w_{dk}$$



Grundannahme

- Vektordimensionen sind von einander unabhängig
- Cluster-Hypothese
 - Dokumente und Anfrage liegen kompakt in einem hochdimensionalen Vektorraum zusammen



Beispiel

- Gewählte Indexterme: „Häuser“, „Italien“, „Gärten“, „Frankreich“
- Grober Ansatz: $w_{dk} = tf_{dk}$
(keine Berücksichtigung der Dokumentlängen)
- $D_1 = \text{„Häuser in Italien“} \Rightarrow V_{D1} = (1; 1; 0; 0)$
- $D_2 = \text{„Häuser in Italien und um Italien“} \Rightarrow V_{D2} = (1; 2; 0; 0)$
- $D_3 = \text{„Gärten und Häuser in Italien“} \Rightarrow V_{D3} = (1; 1; 1; 0)$
- $D_4 = \text{„Gärten in Italien“} \Rightarrow V_{D4} = (0; 1; 1; 0)$
- $D_5 = \text{„Gärten und Häuser in Frankreich“} \Rightarrow V_{D5} = (1; 0; 1; 1)$
- $Q = \text{„Häuser in Italien“} \Rightarrow V_Q = (1; 1; 0; 0)$
- $\text{Similarity}(V_Q, V_{D1}) = 2, \text{Similarity}(V_Q, V_{D2}) = 3, \text{Similarity}(V_Q, V_{D3}) = 2,$
 $\text{Similarity}(V_Q, V_{D4}) = 1, \text{Similarity}(V_Q, V_{D5}) = 1$



5.2 Die Basisformeln zum Vektorraummodell

Berechnung der Termgewichte w_{dk}

- Viele sehr ähnliche Formeln vorgeschlagen – hier die gebräuchlichste Formel
- 2 Faktoren
 1. Vorkommenshäufigkeit des Terms im Dokument
 2. Trennschärfe des Terms bezogen auf Dokumentkollektion



Vorkommenshäufigkeit des Terms im Dokument

- Je höher die **Vorkommenshäufigkeit** tf_{dk} , desto besser beschreibt Term Dokument
- Achtung: absolute Vorkommenshäufigkeit bei langen Dokumenten tendenziell höher
- Konsequenz: lange Dokumente werden bei der Suche bevorzugt
- Beispiel: $V_{D1}=(50;5;0;0)$, $V_{D2}=(0;2;2;0)$, $V_Q=(0;1;1;0)$
 $\Rightarrow V_{D1}$ wird ungerechterweise bevorzugt:
 $\langle V_Q, V_{D1} \rangle = 5$ und $\langle V_Q, V_{D2} \rangle = 4$



Vorkommenshäufigkeit des Terms im Dokument

Unabhängigkeit von Dokumentlänge durch Normierung der Dokumentvektoren:

$$w_{dk} = \frac{tf_{dk}}{\sqrt{\sum_{i=1}^t tf_{di}^2}}$$



Beispiel

- Absolute Häufigkeit: $V_{D1}=(50;5;0;0)$, $V_{D2}=(0;2;2;0)$, $V_Q=(0;1;1;0)$
- Relative Häufigkeit durch Vektornormierung: $V_{D1}=(0,995;0,0995;0;0)$, $V_{D2}=(0;0,707;0,707;0)$
- Anfragevektor unverändert: $V_Q=(0;1;1;0)$
 $\Rightarrow \langle V_Q V_{D1} \rangle = 0,0995$ und $\langle V_Q V_{D2} \rangle = 1,41$
- *davor*: $\langle V_Q V_{D1} \rangle = 5$ und $\langle V_Q V_{D2} \rangle = 4$



Trennschärfe des Terms

- Wenn Term in weniger Dokumenten auftritt, ist er spezifischer
- Spezifischere Terme haben höhere **Trennschärfe** und sollten höher gewichtet werden
- Ansatz: Trennschärfe als Quotient: N/n_i

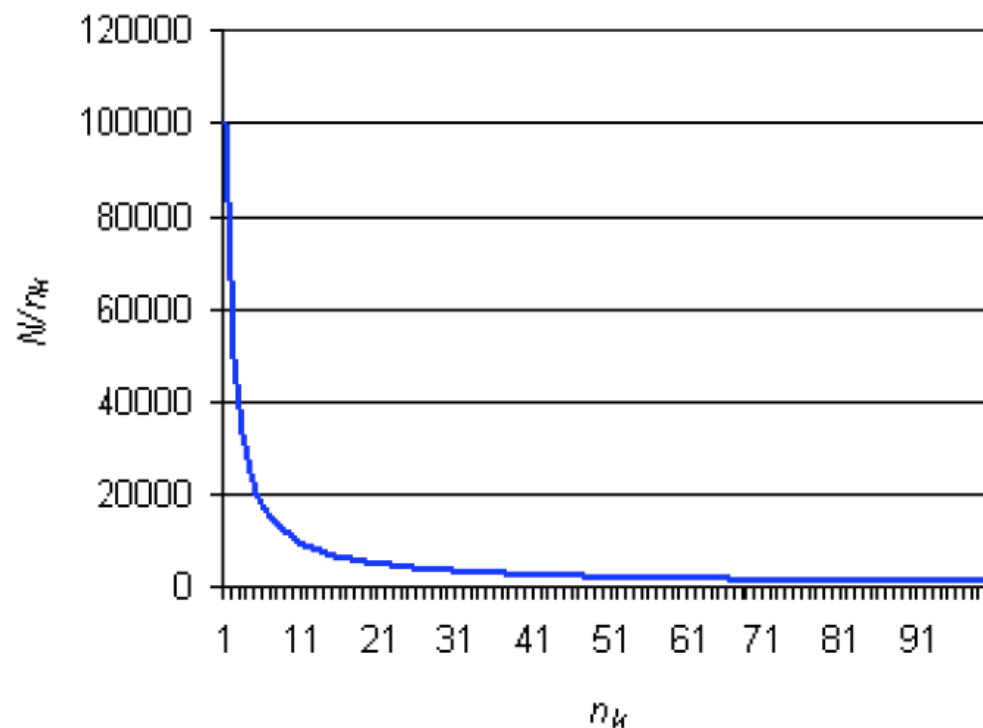
$$w_{dk} = \frac{tf_{dk} \frac{N}{n_k}}{\sqrt{\sum_{i=1}^t \left(tf_{di} \frac{N}{n_i} \right)^2}}$$



Trennschärfe als Quotient

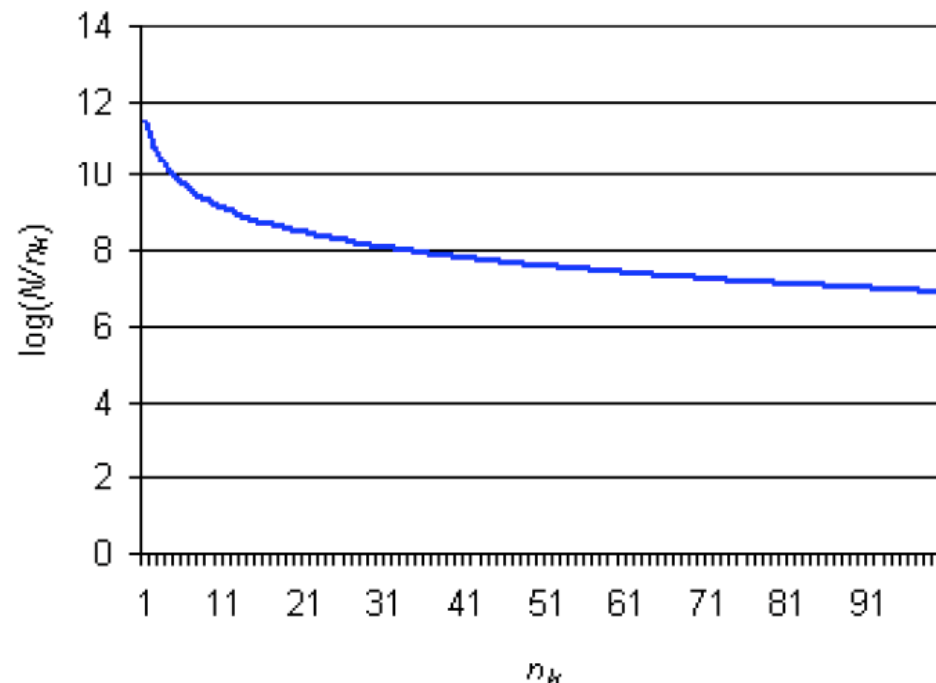
Problem mit N/n_i : sehr seltene Terme dominieren Vektoren

Beispiel: $N=100000$, $n_i=5$ und $n_j=50$



Trennschärfe mittels Logarithmus

- Natürlicher Logarithmus mildert Effekt ab
- $\log(N/n_k)$ statt N/n_k



Die tf·idf-Formel

$$w_{dk} = \frac{tf_{dk} \log \frac{N}{n_k}}{\sqrt{\sum_{i=1}^t \left(tf_{di} \log \frac{N}{n_i} \right)^2}}$$

- *tf* steht für **term frequency**
- *idf* steht für **inverse document frequency**
- tf·idf-Formel ist heuristische Formel



Repräsentation der Anfrage

- Anfrage Q repräsentiert durch $V_Q = (w_{q1}; \dots; w_{qt})$
- w_{qk} -Werte nicht mittels tf-idf-Formel ermitteln, da
 - Jeder Term in Anfrage hat eigenes Grundgewicht 0,5
 - Normierung nicht nötig, da Faktor für alle Dokumente gleich
- Vorschlag von Salton und Buckley:

$$w_{qk} = \begin{cases} \left(0,5 + \frac{0,5 \cdot tf_{qk}}{\max_{1 \leq i \leq t} tf_{qi}} \right) \cdot \log \frac{N}{n_k} & \text{falls } tf_{qk} > 0 \\ 0 & \text{falls } tf_{qk} = 0 \end{cases}$$



Repräsentation der Anfrage

Wie häufig kommt der Term
in der Anfrage vor?

$$w_{qk} = \begin{cases} \left(0,5 + \frac{0,5 \cdot tf_{qk}}{\max_{1 \leq i \leq t} tf_{qi}} \right) \cdot \log \frac{N}{n_k} & \text{falls } tf_{qk} > 0 \\ 0 & \text{falls } tf_{qk} = 0 \end{cases}$$

Trennschärfe

Grundgewicht, da Term
explizit durch Nutzer gewählt wurde

Ähnlichkeit zwischen Dokumenten und Anfrage

- Berechnung über Skalarprodukt:

$$\text{similarity}(V_Q, V_D) = \langle V_Q, V_D \rangle = \sum_{k=1}^t w_{qk} w_{dk}$$

- Gesetz aus linearer Algebra:

$$\langle V_Q, V_D \rangle = \sqrt{\langle V_Q, V_Q \rangle} \cdot \sqrt{\langle V_D, V_D \rangle} \cdot \cos \alpha$$

- Cosinusmaß ist Cosinus des von V_Q und V_D eingeschlossenen Winkels:

$$\text{sim}_{\cos}(V_Q, V_D) = \frac{\sum_{k=1}^t w_{qk} \cdot w_{dk}}{\sqrt{\sum_{i=1}^t w_{qi}^2} \cdot \sqrt{\sum_{j=1}^t w_{dj}^2}}$$

- Erzeugte Dokumentreihenfolge bei Cosinusmaß und Skalarprodukt ist gleich



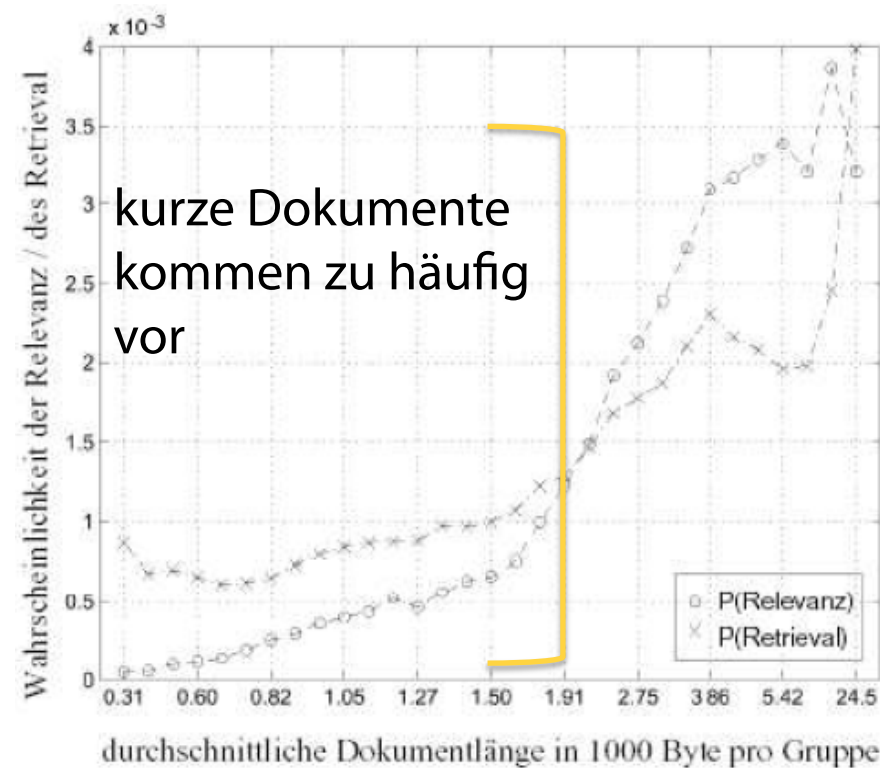
5.3 Eine Variante der tf·idf-Formel

- Viele Verbesserungen der tf·idf-Formel vorgeschlagen
- Im Folgenden exemplarisch eine Variante
- Normierung der Vektoren soll Nachteil kurzer Dokumente vermeiden
- Nebeneffekt: kurze Dokumente haben **wenige** Komponenten mit $w_{dk} > 0$
- Normierung weniger Komponenten erzeugt relativ hohe Werte
- Konsequenz für kurze Anfragen: Bevorzugung kurzer Dokumente
- Normierung ist also „zu viel des Guten“ $\Rightarrow$ Kompensierung



Test an konkretem Datenbestand

CDs 1 und 2 der TREC-Kollektion mit 741856 Dokumenten und Topics 151 bis 200



Anpassung der Normierung

Normierungsfaktor $\sqrt{\sum_{i=1}^t \left(tf_{di} \cdot \log \frac{N}{n_i} \right)^2} = \sqrt{\sum_{i=1}^t w_{di}^2}$

- Abschwächen langer Dokumente
- Verschärfen kurzer Dokumente
- Verwenden von $kf_d w_{dk}$ statt w_{dk} bei Normierung:

$$kf_d = (1 - slope) + slope \cdot \frac{old_normalization}{average_old_normalization}$$



Beispiel

- $slope = 0,75$ und
 $average_old_normalization = 40$

Dokument	Old normalization	kf_d	New normalization	Vektorlänge
D_1	20	0,625	32	0,625
D_2	40	1,0	40	1,0
D_3	80	1,75	45,71	1,75

Bestimmung des slope-Wertes

- Muss empirisch bestimmt werden
- Beispiel CDs 1 und 2 von TREC:
average_old_normalization = 13,36
- Optimal ist slope = 0,75

	<i>tf·idf</i> - Formel	modifizierter Ansatz				
slope	–	0,60	0,65	0,70	0,75	0,80
gefunden	6526	6342	6458	6574	6629	6671
Precision	0,2840	0,3024	0,3097	0,3144	0,3171	0,3162
Verbesserung	–	+6,5%	+9,0%	+10,7%	+11,7%	+11,3%

Zusammenfassung Variante der tf-idf-Formel

- Durch modifizierte Normalisierung Annäherung zwischen Relevanz- und Retrieval-Wahrscheinlichkeit
⇒ höhere Precision
- Achtung: $w_{df} \leq 1$ gilt nicht mehr immer
⇒ Anpassung der Algorithmen, welche auf Normierung vertrauten
- Slope-Wert muss empirisch ermittelt werden



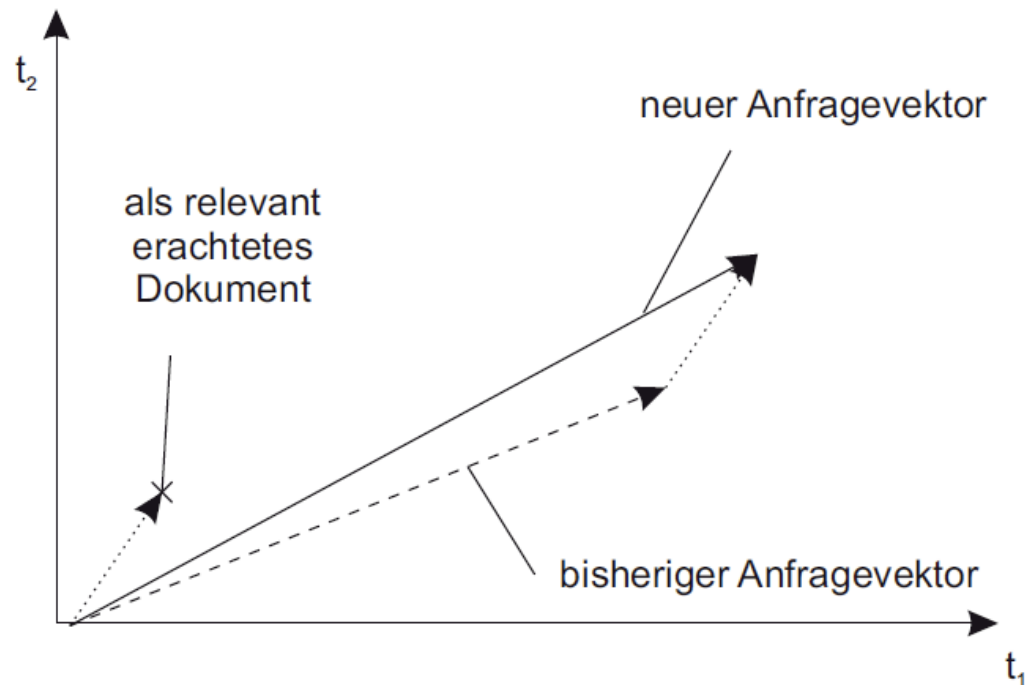
5.4 Relevance Feedback

- Vektorraummodell erwartet vom Nutzer Informationswunsch als Anfrage
- Ziel: Nutzerinteraktionen als weitere Informationsquellen zur Präzisierung nutzen
- Nutzer bewertet nach erstem Durchlauf die einzelnen Ergebnisse (relevant, nicht relevant)
- Relevance Feedback nutzt Bewertung zur Reformulierung neuer Anfrage
- Bewertungen führen zur Verschiebung des Anfragevektors



Verschiebung des Anfragevektors

- 2 Terme
- Ändern des „Suchwinkels“ in Richtung als relevant bewertetes Dokument



Methoden zur Anfrageverschiebung

- Ausgangspunkt ist Zerlegung Feedback-Menge in
 - F^+ (als relevant erachtete Dokumente) und
 - F^- (als irrelevant erachtete Dokumente)
 - Evtl. auch $F^?$ (weder als relevant noch als irrelevant erachtet)
- Methoden
 - Ide (dec hi)
 - Ide (regulär)
 - Rocchio



Ide (dec hi)

- Dokumente aus F^+ werden addiert
- D^{-rel} aus F^- , welches den höchsten Rank im Ergebnis hatte, wird abgezogen

$$V_Q^{neu} = V_Q^{alt} + \left(\sum_{D \in F^+} D \right) - D^{-rel}$$



Ide (regulär)

- Dokumente auf F^+ werden addiert
- Dokumente auf F^- werden abgezogen

$$V_Q^{neu} = V_Q^{alt} + \left(\sum_{D \in F^+} D \right) - \left(\sum_{D \in F^-} D \right)$$



Rocchio

- Dokumente auf F^+ werden gemittelt und addiert
- Dokumente auf F^- werden gemittelt und abgezogen
- Einfluss F^+ und F^- wird mit α bzw. β ($\alpha+\beta=1$) gewichtet
- Typische Belegung: $\alpha=0,25$ und $\beta=0,75$

$$V_Q^{neu} = V_Q^{alt} + \beta \cdot \left(\sum_{D \in F^+} \frac{D}{|F^+|} \right) - \alpha \cdot \left(\sum_{D \in F^-} \frac{D}{|F^-|} \right)$$



Experimentelle Ergebnisse

- Precision-Werte als Mittelwerte über Recall-Stufen 0,25;0,5 und 0,75
- Ergebnisse unter *Ausgewählte Terme*: nur Terme in vielen relevanten Dokumenten berücksichtigt
Vorteil: beschleunigte Anfragebearbeitung
- Erhebliche Verbesserungen der Precision-Werte werden erreicht
- Ide (dec hi) schneidet am besten ab
- Begründung: heterogene Menge F^- , welche zusammen keine klare Richtung angeben



Experimentelle Ergebnisse

eingesetzte Methode		CACM 3204 Dok. 64 Anfr.	CISI 1460 Dok. 112 Anfr.	CRAN 1397 Dok. 225 Anfr.	INSPEC 12684 Dok. 84 Anfr.	MED 1033 Dok. 30 Anfr.	Durchschnitt
initiale Anfrage							
	Precision	0,1459	0,1184	0,1156	0,1368	0,3346	
Ide (dec hi)							
mit allen Termen	Precision	0,2704	0,1742	0,3011	0,2140	0,6305	
	Verbesserung	+86%	+47%	+160%	+56%	+88%	+87%
ausgewählte Terme	Precision	0,2479	0,1924	0,2498	0,1976	0,6218	
	Verbesserung	+70%	+63%	+116%	+44%	+86%	+76%
Ide (regular)							
mit allen Termen	Precision	0,2241	0,1550	0,2508	0,1936	0,6228	
	Verbesserung	+66%	+31%	+117%	+42%	+86%	+68%
ausgewählte Terme	Precision	0,2179	0,1704	0,2217	0,1808	0,5980	
	Verbesserung	+49%	+44%	+92%	+32%	+79%	+59%
Rocchio (standard $\beta = 0,75$; $\alpha = 0,25$)							
mit allen Termen	Precision	0,2552	0,1404	0,2955	0,1821	0,5630	
	Verbesserung	+75%	+19%	+156%	+33%	+68%	+70%
ausgewählte Terme	Precision	0,2491	0,1623	0,2534	0,1861	0,5297	
	Verbesserung	+71%	+37%	+119%	+36%	+55%	+64%

Pseudo-Relevance-Feedback

- Erste Elemente des Rankings werden zum Relevance Feedback verwendet
 - Anfrageerweiterung, damit mehr Recall
 - Funktioniert nur gut, wenn Top- n Precision bereits gut ist



5.5 Implementierung des Vektorraummodells

Varianten:

- Invertierte Listen (Standardansatz)
- Mehrdimensionale Zugriffsstrukturen (Problem der hohen Dimensionalität)
- Signaturen



Basialgorithmen mit invertierten Listen

- Anfragender will meist nur die γ besten Ergebnisse sehen
- Schritte:
 1. Betrachte die invertierten Listen zu den Anfragetermen in der Reihenfolge fallender Anfragetermgewichte w_{qk}
 2. Für jedes Paar (D, w_{dk}) in der gerade betrachteten invertierten Liste addiere $w_{dk} * w_{qk}$ zum bisherigen Gewicht von D
 3. nach vollständiger Betrachtung einer invertierten Liste, prüfe, ob Dokument mit Rang $\gamma+1$ noch Rang γ erreichen kann. Wenn nicht, dann Abbruch

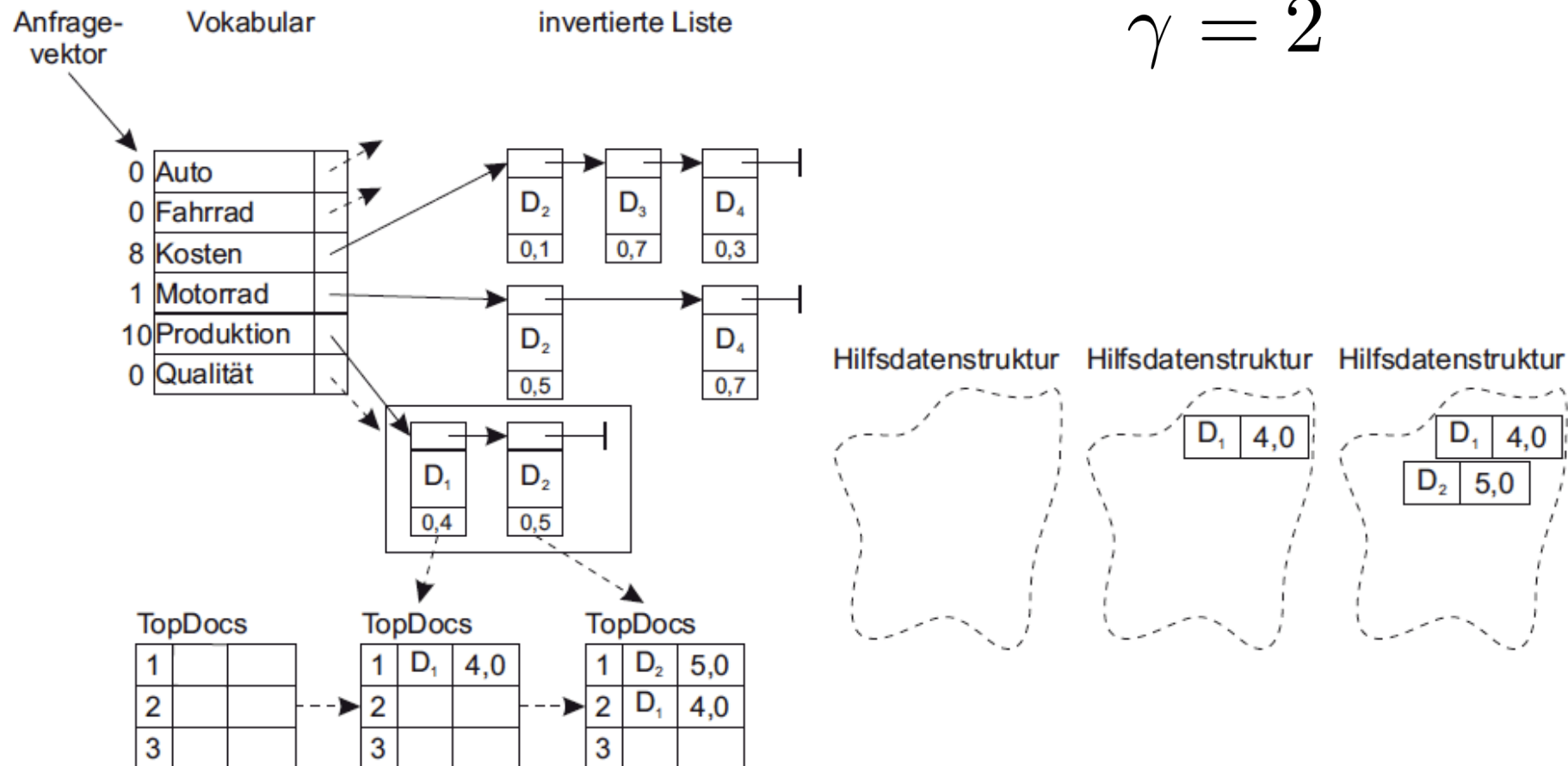


Beispiel

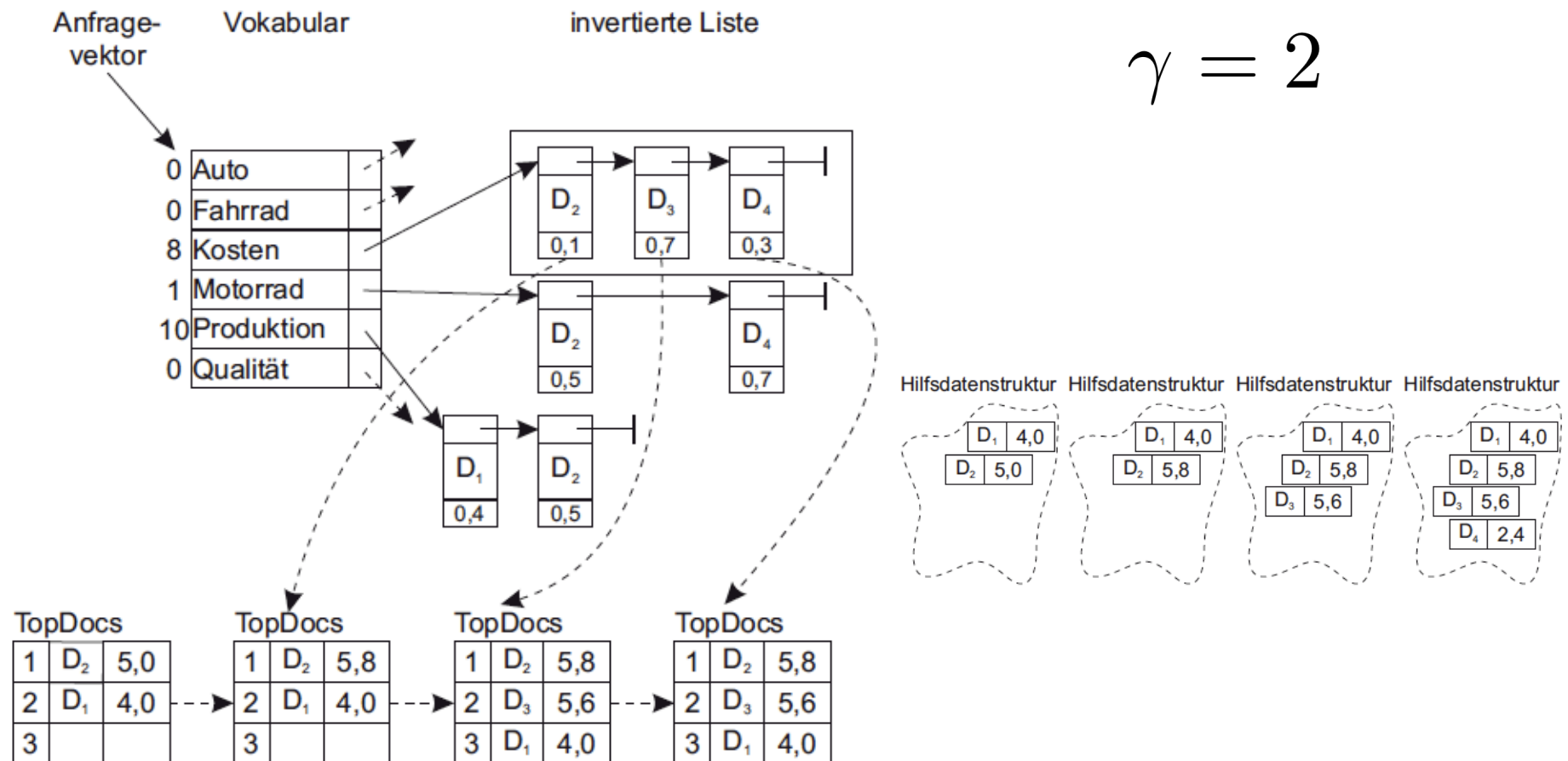
- Vokabular $\{Auto, Fahrrad, Kosten, Motorrad, Produktion, Qualität\}$
- Anfragevektor $(0; 0; 8; 1; 10; 0)$
- $\gamma+1$ momentan besten Dokumente werden im sortierten Array TopDocs gehalten
- Zusätzlich jedes gelesene Dokument in Hilfsdatenstruktur
 - wächst unter Umständen enorm!



Beispiel II



Beispiel III



Realisierung der Hilfsdatenstruktur

- Verwaltung der Paare (D, g) , also Dokument-Id und Gewicht
- Operationen
 - $IsInDS(D)$ testet, ob D enthalten
 - $InsertIntoDS(D, g)$ fügt neues Paar ein
 - $AddToDSEntry(D, \sigma)$ fügt Gewicht σ dem Dokument hinzu
 - $GetWeightFromDS(D)$ ermittelt Gewicht



Notwendige Operationen auf invertierter Liste

Operationen bzgl. Term k :

- *InitList(k)* initialisiert Liste für sequentiellen Durchlauf
- *GetNextElementOfList(k)* liefert aktuelles Paar (D, w_{dk})
- *NotAtEndOfList(k)* testet, ob Ende noch nicht erreicht ist



Realisierung des Array *TopDocs*

- Enthält $\gamma+1$ Dokumente
- Betrachtetes Dokument wird immer richtig einsortiert



Die Operation *MaxRemainingWeight()*

- Ermittelt maximal möglichen Gewichtszuwachs
- Geht von $w_{dk} \leq 1$ aus

$$\textit{MaxRemainingWeight}() = \sum_{\text{alle verbleibenden Anfrageterme } k'} w_{qk'}$$



Algorithmus nach Buckley & Lewit

- Liefert γ relevantesten Dokumente (aber nicht unbedingt in richtiger Reihenfolge)

```

for (alle Terme im Anfragevektor mit  $w_{qk} > 0$ ) do
  Sei  $k'$  der bisher noch nicht betrachtete Term mit dem
    höchsten Wert  $w_{qk}$ 
  InitList( $k'$ );
  while (NotAtEndOfList( $k'$ )) do /* durchlaufe die Liste */
    ( $D, w_{dk'}$ ) = GetNextElemOfList( $k'$ );
    if IsInDS( $D$ ) then
      AddToDSEntry( $D, w_{qk'} \cdot w_{dk'}$ );
    else
      InsertIntoDS( $D, w_{qk'} \cdot w_{dk'}$ );
    end;
    Sortiere  $D$  im Array TopDocs gemäß dem Gewicht
      GetWeightFromDS( $D$ ) ein;
  end;
  if (TopDocs[ $\gamma$ ] > TopDocs[ $\gamma + 1$ ] + MaxRemainingWeight())
    then FINISH;
end;

```

Steigerung der Performance

- Idee: Algorithmus früher abbrechen lassen und Ungenauigkeit in Kauf nehmen
- Buckley & Lewitt: Abbruch wenn $n \leq \gamma$
Dokumente sicher erkannt wurden

```
if (TopDocs[n] > TopDocs[ $\gamma + 1$ ] + MaxRemainingWeight()) then  
    FINISH;
```



Experimente mit $\gamma=10$

n	CPU-Zeit	# gelesene Blöcke	Recall
1	61.3	2833	0.1588
2	68.2	3058	0.1566
3	75.9	3255	0.1555
4	83.1	3451	0.1538
5	90.3	3630	0.1548
6	93.6	3721	0.1543
7	99.7	3863	0.1501
8	104.5	3980	0.1505
9	107.0	4033	0.1508
10	109.2	41	0.1508

Besseres
Ergebnis
trotz frühen
Abbruchs

Baseline

Auswertung der Experimente

- Je kleiner n desto besser der Recall-Wert
- Möglicher Grund:
 - Hochgewichtete Terme (hohe Trennschärfe) werden berücksichtigt, während bei langen Anfragen die restlichen Terme das Ergebnis „verwässern“
- Fazit: exakte Berechnung führt nicht unbedingt zu besseren Ergebnissen



Bewertung Vektorraummodell

■ Vorteile des Modells:

- Einfachheit
- Im Gegensatz zum probabilistischen Modell unmittelbar auf Dokumentkollektion anwendbar
- Sehr gute empirische Retrieval-Qualität
- Effiziente Implementierung durch invertierte Listen
- Relevance Feedback verbessert Ergebnisqualität

■ Nachteile:

- Geht von Unabhängigkeit der Dimensionen aus
- Keine theoretische Fundierung des Modells
- Anpassung an strukturierte Dokumente nur schwer möglich



Bewertung Vektorraummodell

■ Vorteile des Modells:

- Einfachheit
- Im Gegensatz zum probabilistischen Modell unmittelbar auf Dokumentkollektion anwendbar
- Sehr gute empirische Retrieval-Qualität
- Effiziente Implementierung durch invertierte Listen
- Relevance Feedback verbessert Ergebnisqualität

■ Nachteile:

- Geht von Unabhängigkeit der Dimensionen aus
- Keine theoretische Fundierung des Modells
- Anpassung an strukturierte Dokumente nur schwer möglich



Bewertung Vektorraummodell

■ Vorteile des Modells:

- Einfachheit
- Im Gegensatz zum probabilistischen Modell unmittelbar auf Dokumentkollektion anwendbar
- Sehr gute empirische Retrieval-Qualität
- Effiziente Implementierung durch invertierte Listen
- Relevance Feedback verbessert Ergebnisqualität

■ Nachteile:

- Geht von Unabhängigkeit der Dimensionen aus
- Keine theoretische Fundierung des Modells
- Anpassung an strukturierte Dokumente nur schwer möglich

